



Questões Práticas

1. Divisão Segura com try-catch

Crie um programa em Java que possua um método chamado `dividir`.

Esse método deve receber dois números inteiros:

```
public static int dividir(int a, int b)
```

O método deve tentar dividir o valor de `a` pelo valor de `b`.

Caso o valor de `b` seja igual a zero, o programa deve tratar a exceção `ArithmeticException` e exibir a seguinte mensagem:

```
Erro: nao e possivel dividir por zero.
```

Nesse caso, o método deve retornar o valor 0.

No método `main`, teste o método `dividir` com uma divisão válida e com uma divisão por zero.

Requisitos:

- criar o método `dividir(int a, int b)`;
- utilizar `try-catch`;
- tratar a exceção `ArithmeticException`;
- retornar 0 quando ocorrer divisão por zero;
- mostrar que o programa continua funcionando após o erro.

2. Validação de Idade com NumberFormatException e throw

Crie um programa que leia a idade de uma pessoa pelo teclado usando `Scanner`.

A idade deve ser lida inicialmente como `String` e depois convertida para `int` usando:

```
int idade = Integer.parseInt(entrada);
```

Caso o usuário digite um valor inválido, como "abc" ou "vinte", o programa deve tratar a exceção `NumberFormatException`.

Caso o usuário digite uma idade negativa, o programa deve lançar manualmente uma exceção usando:

```
throw new IllegalArgumentException("Idade nao pode ser negativa");
```

Requisitos:

- a) Ler a idade usando `Scanner`;
- b) Converter a entrada usando `Integer.parseInt()`;
- c) Tratar `NumberFormatException`;
- d) Usar `throw` para impedir idade negativa;
- e) Tratar `IllegalArgumentException`;
- f) Exibir mensagens claras para o usuário.

Exemplos de saída esperada:

```
Digite sua idade: vinte
Erro: idade deve ser um numero inteiro.
```

```
Digite sua idade: -5
Erro: idade nao pode ser negativa.
```

```
Digite sua idade: 20
Idade cadastrada com sucesso: 20 anos.
```

3. Conta Bancária com Exceção Personalizada

Crie um pequeno sistema bancário utilizando tratamento de exceções.

O sistema deve possuir as seguintes classes:

```
ContaBancaria
SaldoInsuficienteException
Main
```

A classe `ContaBancaria` deve possuir o seguinte atributo:

```
private double saldo;
```

E os seguintes métodos:

```
public void depositar(double valor)
public void sacar(double valor) throws
    SaldoInsuficienteException
public double getSaldo()
```

A classe `SaldoInsuficienteException` deve ser uma exceção personalizada do tipo *checked*, ou seja, deve herdar de `Exception`.

```
public class SaldoInsuficienteException extends Exception {

}
```

O método `sacar` deve lançar essa exceção quando o valor do saque for maior que o saldo disponível.

Requisitos:

- a) Criar uma exceção personalizada chamada `SaldoInsuficienteException`;

- b) Usar `extends Exception`;
- c) Usar `throw` dentro do método `sacar`;
- d) Usar `throws` na assinatura do método `sacar`;
- e) Tratar a exceção na classe `Main` com `try-catch`;
- f) Exibir o saldo atual após cada operação.

Exemplo de saída esperada:

```
Saldo inicial: R$ 100.0
Tentando sacar R$ 150.0...

Erro: saldo insuficiente.
Saldo disponível: R$ 100.0

Programa finalizado normalmente.
```

Desafio extra:

Adicione uma validação para impedir depósito ou saque com valor menor ou igual a zero. Para isso, utilize `IllegalArgumentException`.